Windows 10 C++車両制御開発環境 構築マニュアル

MinGW-w64・CMake・Eigen・OSQP・Python

用途: 車両運動シミュレーション、線形MPC、自作SQPによる非線形MPC
対象OS: Windows 10 64ビット版
開発言語: C++17、Python 3
確認済み基本構成: MinGW-w64 GCC 8.2.0、CMake、Eigen 3.4.1、OSQP 1.0.0

目次

1. 本マニュアルの目的
2. 完成する開発環境
3. 各ツールの役割
4. 重要なフォルダー規則
5. 事前確認
6. VS Codeの準備
7. MinGW-w64 GCCの確認
8. CMakeの導入と確認
9. MinGW-w64とCMakeの結合確認
10. Eigenの導入
11. Eigenの単体確認
12. OSQPの導入
13. OSQPの単体確認
14. EigenとOSQPの結合確認
15. Python可視化環境
16. 自動駐車プロジェクトの推奨構成
17. CMake Presetsによる設定管理
18. Debug・Releaseビルド
19. Gitによるバージョン管理
20. よくあるエラーと対処
21. 自作SQP型NMPCに必要な環境
22. 将来の環境更新方針
23. 最終チェックリスト
24. 公式参考資料

---

1. 本マニュアルの目的

本マニュアルでは、Windows 10上に次の開発環境を構築します。

- C++で車両モデルと制御アルゴリズムを実装する
- CMakeでC++プロジェクトを管理する
- Eigenで行列・ベクトル演算を行う

- OSQPで二次計画問題を解く
- 線形MPCをC++で実装する
- EigenとOSQPを使い、自作SQP型NMPCへ発展させる
- C++の計算結果をCSVへ出力する
- Pythonでグラフとアニメーションを作成する

現段階では、次のツールは必須ではありません。

- Visual Studio Build Tools
- MSVC
- MSYS2 UCRT64
- Ninja
- CasADi
- IPOPT
- Boost
- ROS

これらは具体的な必要性が生じた段階で追加します。

---

2. 完成する開発環境

```
VS Code
   ├─ C/C++ソース編集
   ├─ Pythonソース編集
   └─ PowerShellターミナル
          ↓
CMake
   └─ MinGW Makefilesを生成
          ↓
mingw32-make
   └─ 必要なビルド処理を実行
          ↓
MinGW-w64 GCC
   ├─ gcc：Cコンパイラ
   └─ g++：C++コンパイラ
          ↓
C++実行ファイル
   ├─ Eigen：行列演算
   ├─ OSQP：QP求解
   └─ CSV出力
          ↓
Python
   ├─ NumPy：数値データ処理
   └─ Matplotlib：可視化
```

---

3. 各ツールの役割

ツール	役割
VS Code	ソース編集、ターミナル、デバッグ操作
MinGW-w64 GCC	C/C++をWindows実行ファイルへコンパイル
`mingw32-make`	CMakeが生成したMakefileを実行
CMake	ビルド対象、オプション、ライブラリ依存を管理
Eigen	行列・ベクトル、密行列・疎行列演算
OSQP	凸二次計画問題を数値的に解く
Python	C++結果の解析・可視化
Git	ソースコードと設定の変更履歴を管理

重要な関係は次です。

```
CMakeはコンパイラではない。
CMakeだけではC++を実行ファイルにできない。
```

```
CMake
  → mingw32-make
  → g++
  → .exe
```

また、Eigenは最適化ソルバーではありません。

```
Eigen: QP行列を作る
OSQP: 作られたQPを解く
```

---

4. 重要なフォルダー規則

現在使用している比較的古いMinGW環境では、日本語パスがMakefile上で文字化けし、ビルドに失敗することがあります。

実際に発生した例は次です。

```
No rule to make target 'C:/Users/.../2026_自動駐車/.../main.cpp'
```

そのため、開発パスは半角英数字とアンダースコアだけにします。

推奨パス

```
C:\work\auto_parking_sim
C:\dev\libs
```

避けるパス

```
C:\Users\...\Desktop\自動駐車
C:\Users\...\OneDrive\車両制御
C:\My Projects\auto parking
```

次の規則を推奨します。

- 日本語を使わない
- 空白を使わない
- パスを長くしすぎない
- OneDrive配下をできるだけ避ける
- ソース、ビルド、ライブラリの場所を分ける

---

5. 事前確認

新しいPowerShellまたはVS Codeターミナルで、次を実行します。

```
g++ --version
g++ -v
mingw32-make --version
cmake --version
git --version
python --version
```

実体の場所を確認します。

```
where.exe g++
where.exe gcc
where.exe mingw32-make
where.exe cmake
where.exe git
where.exe python
```

MinGW-w64の64ビット環境では、`g++ -v`に概ね次が含まれます。

```
Target: x86_64-w64-mingw32
```

確認済み環境では、CMake設定時に次が表示されました。

```
The CXX compiler identification is GNU 8.2.0
```

---

6. VS Codeの準備

VS Code公式サイトからWindows版を導入します。

推奨拡張機能は次です。

拡張機能	発行元	用途
C/C++	Microsoft	C++補完・GDBデバッグ
CMake Tools	Microsoft	CMake設定・ビルド
CMake	twxs	CMakeLists.txt編集支援
Python	Microsoft	Python実行・デバッグ
Pylance	Microsoft	Python補完

VS Codeのターミナルは、次から開けます。

```
メニュー → ターミナル → 新しいターミナル
```

PowerShellになっていることを確認します。

---

7. MinGW-w64 GCCの確認

既にMinGWが入っている場合、再インストールは不要です。次が成功することを確認します。

```
g++ --version
```

```
mingw32-make --version
```

```
where.exe g++
where.exe mingw32-make
```

`g++`はC++コンパイラ、`mingw32-make`はビルド実行ツールです。

GDBが入っているかも確認します。

```
gdb --version
```

GDBがなくてもコンパイル・実行はできますが、ブレークポイントやステップ実行には必要です。

---

8. CMakeの導入と確認

CMake公式ダウンロードページから、Windows x64用MSIを取得します。

インストール時にPATH追加を有効にします。

```
Add CMake to the system PATH
```

インストール後、VS CodeとPowerShellを完全に開き直します。

確認：

```
cmake --version
where.exe cmake
```

---

9. MinGW-w64とCMakeの結合確認

9.1 フォルダー作成

```
New-Item -ItemType Directory -Force C:\work\cmake_test
Set-Location C:\work\cmake_test
```

9.2 `main.cpp`

```
#include <iostream>
```

```
int main()
{
    std::cout << "MinGW-w64 GCC and CMake are working.\n";
    return 0;
}
```

9.3 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)

project(cmake_test LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

add_executable(cmake_test main.cpp)

target_compile_options(
    cmake_test
    PRIVATE
    -Wall
    -Wextra
    -Wpedantic
)
```

9.4 CMake設定

```
cmake -S . -B build `
    -G "MinGW Makefiles" `
    -DCMAKE_BUILD_TYPE=Debug
```

9.5 ビルドと実行

```
cmake --build build
```

```
.\build\cmake_test.exe
```

次が表示されれば成功です。

```
MinGW-w64 GCC and CMake are working.
```

---

10. Eigenの導入

10.1 ライブラリ用フォルダー

```
New-Item -ItemType Directory -Force C:\dev\libs\src
New-Item -ItemType Directory -Force C:\dev\libs\install
Set-Location C:\dev\libs\src
```

10.2 Eigen 3.4.1を取得

```
git clone `
    --branch 3.4.1 `
    --depth 1 `
    https://gitlab.com/libeigen/eigen.git `
    eigen-3.4.1
```

10.3 Eigenをインストール

```
Set-Location C:\dev\libs\src\eigen-3.4.1
```

```
cmake -S . -B build `
    -G "MinGW Makefiles" `
    -DCMAKE_BUILD_TYPE=Release `
    -DCMAKE_INSTALL_PREFIX=C:/dev/libs/install/eigen `
    -DBUILD_TESTING=OFF
```

```
cmake --install build
```

確認：

```
Test-Path C:\dev\libs\install\eigen\include\eigen3\Eigen\Dense
```

Trueなら成功です。

```
Get-ChildItem C:\dev\libs\install\eigen `
    -Recurse `
    -Filter Eigen3Config.cmake
```

---

11. Eigenの単体確認

11.1 main.cpp

```
#include <Eigen/Dense>
#include <iostream>

int main()
{
    Eigen::Matrix2d matrix;
    matrix << 1.0, 2.0,
              3.0, 4.0;

    Eigen::Vector2d vector;
    vector << 5.0, 6.0;

    const Eigen::Vector2d result = matrix * vector;

    std::cout << result << '\n';
    return 0;
}
```

11.2 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)

project(eigen_test LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

find_package(Eigen3 3.4 REQUIRED NO_MODULE)

add_executable(eigen_test main.cpp)

target_link_libraries(
    eigen_test
    PRIVATE
```

```
    Eigen3::Eigen
)
```

11.3 ビルド

```
cmake -S . -B build `
    -G "MinGW Makefiles" `
    -DCMAKE_BUILD_TYPE=Debug `
    -DCMAKE_PREFIX_PATH=C:/dev/libs/install/eigen
```

```
cmake --build build
.\build\eigen_test.exe
```

結果：

```
17
39
```

---

12. OSQPの導入

12.1 OSQP 1.0.0を取得

```
Set-Location C:\dev\libs\src
```

```
git clone `
    --branch v1.0.0 `
    --depth 1 `
    --recursive `
    https://github.com/osqp/osqp.git `
    osqp
```

`--recursive`は依存ソースも取得するために必要です。

12.2 CMake設定

```
Set-Location C:\dev\libs\src\osqp
```

```
cmake -S . -B build `
    -G "MinGW Makefiles" `
    -DCMAKE_BUILD_TYPE=Release `
    -DCMAKE_INSTALL_PREFIX=C:/dev/libs/install/osqp
```

12.3 ビルド・インストール

```
cmake --build build
cmake --install build
```

12.4 インストール結果確認

```
Get-ChildItem C:\dev\libs\install\osqp `
    -Recurse `
    -Filter osqp.h
```

```
Get-ChildItem C:\dev\libs\install\osqp `
    -Recurse `
    -Include *.a,*.dll
```

```
Get-ChildItem C:\dev\libs\install\osqp `
    -Recurse `
    -Include "*Config.cmake","*config.cmake"
```

---

13. OSQPの単体確認

OSQPのビルドフォルダーに生成された実行ファイルを検索します。

```
Get-ChildItem C:\dev\libs\src\osqp\build `
    -Recurse `
    -Filter "*.exe"
```

`osqp_demo.exe`がある場合、表示されたフルパスで実行します。

例：

```
C:\dev\libs\src\osqp\build\out\osqp_demo.exe
```

最適化結果が表示され、正常終了すればOSQP単体ビルドは成功です。

---

14. EigenとOSQPの結合確認

14.1 テストQP

次を解きます。

$$\min_x \frac{1}{2} x^T \begin{bmatrix} 4 & 1 \\ 1 & 2 \end{bmatrix} x + \begin{bmatrix} 1 \\ 1 \end{bmatrix}^T x$$

制約：

$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \leq \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} x \leq \begin{bmatrix} 1 \\ 0.7 \\ 0.7 \end{bmatrix}$$

最適解は概ね、

$$x^* = \begin{bmatrix} 0.3 \\ 0.7 \end{bmatrix}$$

です。

14.2 フォルダー

```
New-Item -ItemType Directory -Force C:\work\eigen_osqp_test
Set-Location C:\work\eigen_osqp_test
```

14.3 main.cpp

```
#include <Eigen/Dense>
#include <osqp.h>

#include <cmath>
#include <iomanip>
#include <iostream>

int main()
{
    Eigen::Matrix2d P_dense;
    P_dense << 4.0, 1.0,
               1.0, 2.0;
```

```
    Eigen::Vector2d q_eigen;
    q_eigen << 1.0, 1.0;

    OSQPFloat P_x[3] = {4.0, 1.0, 2.0};
    OSQPInt P_i[3] = {0, 0, 1};
    OSQPInt P_p[3] = {0, 1, 3};

    OSQPFloat A_x[4] = {1.0, 1.0, 1.0, 1.0};
    OSQPInt A_i[4] = {0, 1, 0, 2};
    OSQPInt A_p[3] = {0, 2, 4};

    OSQPFloat q[2] = {
        static_cast<OSQPFloat>(q_eigen(0)),
        static_cast<OSQPFloat>(q_eigen(1))
    };

    OSQPFloat l[3] = {1.0, 0.0, 0.0};
    OSQPFloat u[3] = {1.0, 0.7, 0.7};

    constexpr OSQPInt n = 2;
    constexpr OSQPInt m = 3;

    OSQPCscMatrix* P = OSQPCscMatrix_new(
        n, n, 3, P_x, P_i, P_p);

    OSQPCscMatrix* A = OSQPCscMatrix_new(
        m, n, 4, A_x, A_i, A_p);

    OSQPSettings* settings = OSQPSettings_new();
    settings->alpha = 1.0;
    settings->polishing = 1;
    settings->verbose = 1;

    OSQPSolver* solver = nullptr;

    OSQPInt exit_flag = osqp_setup(
        &solver, P, q, A, l, u, m, n, settings);

    if (exit_flag == 0)
    {
        exit_flag = osqp_solve(solver);
    }

    if (
        (exit_flag != 0)
        || (solver == nullptr)
        || (solver->solution == nullptr)
        || (solver->solution->x == nullptr)
    )
    {
        std::cerr << "OSQP solve failed.\n";

        if (solver != nullptr)
```

```
        {
            osqp_cleanup(solver);
        }

        OSQPSettings_free(settings);
        OSQPCscMatrix_free(A);
        OSQPCscMatrix_free(P);

        return 1;
    }

    const double x0 = solver->solution->x[0];
    const double x1 = solver->solution->x[1];

    std::cout << std::fixed << std::setprecision(9);
    std::cout << "x[0] = " << x0 << '\n';
    std::cout << "x[1] = " << x1 << '\n';

    const bool passed =
        std::abs(x0 - 0.3) < 1.0e-4
        && std::abs(x1 - 0.7) < 1.0e-4;

    std::cout
        << (passed
            ? "Eigen + OSQP integration test: PASSED\n"
            : "Eigen + OSQP integration test: FAILED\n");

    osqp_cleanup(solver);
    OSQPSettings_free(settings);
    OSQPCscMatrix_free(A);
    OSQPCscMatrix_free(P);

    return passed ? 0 : 1;
}
```

14.4 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)

project(eigen_osqp_test LANGUAGES C CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

find_package(Eigen3 3.4 REQUIRED NO_MODULE)
find_package(osqp REQUIRED CONFIG)

if(TARGET osqp::osqpstatic)
    set(OSQP_LINK_TARGET osqp::osqpstatic)
elseif(TARGET osqp::osqp)
```

```
    set(OSQP_LINK_TARGET osqp::osqp)
else()
    message(FATAL_ERROR "OSQP CMake target was not found.")
endif()

add_executable(eigen_osqp_test main.cpp)

target_link_libraries(
    eigen_osqp_test
    PRIVATE
    Eigen3::Eigen
    ${OSQP_LINK_TARGET}
)

target_compile_options(
    eigen_osqp_test
    PRIVATE
    -Wall
    -Wextra
    -Wpedantic
)
```

14.5 設定・ビルド・実行

```
cmake -S . -B build `
    -G "MinGW Makefiles" `
    -DCMAKE_BUILD_TYPE=Debug `
    -DCMAKE_PREFIX_PATH="C:/dev/libs/install/eigen;C:/dev/libs/install/osqp"
```

```
cmake --build build
.\build\eigen_osqp_test.exe
```

次が表示されれば環境構築完了です。

```
x[0] = 0.300000000
x[1] = 0.700000000
Eigen + OSQP integration test: PASSED
```

---

15. Python可視化環境

15.1 仮想環境作成

```
Set-Location C:\work\auto_parking_sim
python -m venv .venv
```

15.2 有効化

```
.\.venv\Scripts\Activate.ps1
```

PowerShellの実行ポリシーで拒否された場合、個人PCでは必要に応じて次を使用します。

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

会社管理PCでは管理規則に従ってください。代わりにコマンドプロンプトから次を使えます。

```
.venv\Scripts\activate.bat
```

15.3 ライブラリ導入

```
python -m pip install --upgrade pip
python -m pip install numpy matplotlib
```

確認：

```
python -c "import numpy; import matplotlib; print('Python OK')"
```

依存関係保存：

```
python -m pip freeze > requirements.txt
```

---

16. 自動駐車プロジェクトの推奨構成

```
auto_parking_sim/
├─ CMakeLists.txt
├─ CMakePresets.json
├─ README.md
```

```
├─ requirements.txt
├─ .gitignore
├─ include/
│  ├─ vehicle_model.hpp
│  ├─ path_generator.hpp
│  ├─ path_tracker.hpp
│  ├─ linear_mpc.hpp
│  ├─ sqp_solver.hpp
│  ├─ qp_solver_interface.hpp
│  └─ osqp_solver.hpp
├─ src/
│  ├─ main.cpp
│  ├─ vehicle_model.cpp
│  ├─ path_generator.cpp
│  ├─ path_tracker.cpp
│  ├─ linear_mpc.cpp
│  ├─ sqp_solver.cpp
│  └─ osqp_solver.cpp
├─ test/
│  ├─ test_vehicle_model.cpp
│  ├─ test_jacobian.cpp
│  ├─ test_qp.cpp
│  └─ test_sqp.cpp
├─ visualization/
│  ├─ plot_simulation.py
│  └─ animate_vehicle.py
├─ config/
├─ results/
├─ build-debug/
├─ build-release/
└─ .venv/
```

---

17. CMake Presetsによる設定管理

プロジェクトルートに`CMakePresets.json`を作成します。

```
{
  "version": 3,
  "configurePresets": [
    {
      "name": "mingw-debug",
      "displayName": "MinGW Debug",
      "generator": "MinGW Makefiles",
      "binaryDir": "${sourceDir}/build-debug",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "CMAKE_PREFIX_PATH": "C:/dev/libs/install/eigen;C:/dev/libs/install/osqp"
      }
    },
```

```
    {
      "name": "mingw-release",
      "displayName": "MinGW Release",
      "generator": "MinGW Makefiles",
      "binaryDir": "${sourceDir}/build-release",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Release",
        "CMAKE_PREFIX_PATH": "C:/dev/libs/install/eigen;C:/dev/libs/install/osqp"
      }
    }
  ],
  "buildPresets": [
    {
      "name": "mingw-debug",
      "configurePreset": "mingw-debug"
    },
    {
      "name": "mingw-release",
      "configurePreset": "mingw-release"
    }
  ]
}
```

使用方法：

```
cmake --preset mingw-debug
cmake --build --preset mingw-debug
```

---

18. Debug・Releaseビルド

MinGW Makefilesは単一構成ジェネレーターです。DebugとReleaseでビルドフォルダーを分けます。

Debug

```
cmake -S . -B build-debug `
    -G "MinGW Makefiles" `
    -DCMAKE_BUILD_TYPE=Debug

cmake --build build-debug
```

Release

```
cmake -S . -B build-release `
    -G "MinGW Makefiles" `
```

```
    -DCMAKE_BUILD_TYPE=Release

cmake --build build-release
```

アルゴリズム開発中はDebug、性能測定はReleaseを使用します。

---

19. Gitによるバージョン管理

.gitignore

```
build/
build-debug/
build-release/
.venv/
__pycache__/
*.pyc
.vscode/
results/*.csv
results/*.png
results/*.mp4
Thumbs.db
Desktop.ini
```

初期化

```
git init
git add .
git commit -m "Initial vehicle control development environment"
```

ライブラリのビルド生成物やPython仮想環境はGit管理しません。導入バージョンと手順をREADMEへ記録します。

---

20. よくあるエラーと対処

20.1 日本語パスでMakeが失敗する

症状：

```
No rule to make target ...
```

対処：

```
C:\work\project_name
```

へソースを移します。既存`build`はコピーせず、新しい場所で作り直します。CMakeキャッシュには絶対パスが保存されるためです。

20.2 CMake generatorが一致しない

症状：

```
generator does not match the generator used previously
```

対処：新しいビルドフォルダーを使います。

```
cmake -S . -B build-mingw -G "MinGW Makefiles"
```

20.3 Eigenが見つからない

```
Could not find Eigen3Config.cmake
```

検索：

```
Get-ChildItem C:\dev\libs\install\eigen `
    -Recurse `
    -Filter Eigen3Config.cmake
```

設定時：

```
-DCMAKE_PREFIX_PATH=C:/dev/libs/install/eigen
```

20.4 OSQPが見つからない

```
Could not find a package configuration file provided by osqp
```

検索：

```
Get-ChildItem C:\dev\libs\install\osqp `
    -Recurse `
    -Include "*Config.cmake","*config.cmake"
```

設定時：

```
-DCMAKE_PREFIX_PATH="C:/dev/libs/install/eigen;C:/dev/libs/install/osqp"
```

20.5 OSQP依存ソースがない

```
Set-Location C:\dev\libs\src\osqp
git submodule update --init --recursive
```

その後、新しいビルドフォルダーを使います。

20.6 DLLが見つからない

共有OSQPをリンクした場合、実行時にDLLが必要です。

```
Get-ChildItem C:\dev\libs\install\osqp `
    -Recurse `
    -Filter "*.dll"
```

一時的なPATH設定：

```
$env:Path = "C:\dev\libs\install\osqp\bin;" + $env:Path
```

今回のCMakeでは、利用可能なら`osqp::osqpstatic`を優先します。

20.7 `<filesystem>`でリンクエラーになる

GCC 8では、環境によって`std::filesystem`に追加リンクが必要です。

```
if(
    CMAKE_CXX_COMPILER_ID STREQUAL "GNU"
    AND CMAKE_CXX_COMPILER_VERSION VERSION_LESS 9
)
    target_link_libraries(
        your_target
        PRIVATE
        stdc++fs
```

```
    )
endif()
```

長期的には新しいGCCへの更新を推奨します。

---

21. 自作SQP型NMPCに必要な環境

自作SQPでは、追加の必須ライブラリはありません。

```
C++標準ライブラリ
    └ 非線形車両モデル、反復処理、収束判定

Eigen
    └ ヤコビ行列、QP行列、ベクトル演算

OSQP
    └ 各SQP反復の凸QPを求解
```

SQPの処理は次です。

```
1. 非線形モデルで予測
2. 予測軌道周りで線形化
3. QPを構築
4. OSQPでQPを解く
5. 予測状態・入力を更新
6. 収束するまで反復
7. 最初の入力だけ車両へ出力
```

OSQPが扱うQPの二次行列は正半定値である必要があります。最初は、Gauss–Newton近似と小さな正則化を使用します。

$$H \approx J_r^T W J_r$$

$$H_{reg} = H + \rho I$$

任意追加候補は次です。

ツール	用途
GoogleTest・Catch2	単体テスト
clang-format	コード整形
clang-tidy・Cppcheck	静的解析
CppAD・autodiff	複雑なモデルの自動微分

---

22. 将来の環境更新方針

現在のGCC 8.2は、初期シミュレーションと自作SQPの学習には使用できますが、長期的には古い環境です。

次の段階で更新を検討します。

- 新しいOSQPへ更新する
- 複雑な外部ライブラリを導入する
- C++20以降を使用する
- 実行速度を本格評価する
- 実車向け環境とコンパイラ差を確認する

更新候補：

```
MSYS2 UCRT64
├─ 新しいMinGW-w64 GCC
├─ GDB
├─ CMake
└─ NinjaまたはMake
```

更新時は、Eigen、OSQP、自動駐車プロジェクトを同じ新しいコンパイラで再ビルドします。旧環境と新環境の`g++`をPATH上で無秩序に混在させないでください。

実車段階では、対象ECU用クロスコンパイラで制御コアを再ビルドします。QPソルバーを交換できるよう、次のように依存関係を分離します。

```
NMPC問題
   ↓
SQPアルゴリズム
   ↓
QPソルバーインターフェース
   ↓
OSQP
```

---

23. 最終チェックリスト

基本ツール

- [ ] `g++ --version`が成功する
- [ ] `g++ -v`で64ビットMinGW-w64を確認できる
- [ ] `mingw32-make --version`が成功する
- [ ] `cmake --version`が成功する
- [ ] `git --version`が成功する
- [ ] プロジェクトパスに日本語・空白がない

MinGW＋CMake

- [ ] `cmake -S . -B build -G "MinGW Makefiles"`が成功する
- [ ] `cmake --build build`が成功する
- [ ] `.exe`を実行できる

Eigen

- [ ] `Eigen3Config.cmake`を確認できる
- [ ] `find_package(Eigen3)`が成功する
- [ ] 行列積の結果が`17, 39`になる

OSQP

- [ ] OSQP本体をビルドできる
- [ ] OSQPを`C:\dev\libs\install\osqp`へインストールできる
- [ ] `osqp.h`を確認できる
- [ ] `find_package(osqp)`が成功する
- [ ] 付属デモを実行できる

Eigen＋OSQP

- [ ] CMakeで両方を同時検出できる
- [ ] C++からOSQPを呼び出せる
- [ ] QP解が`x=[0.3, 0.7]`になる
- [ ] `Eigen + OSQP integration test: PASSED`と表示される

Python

- [ ] `.venv`を作成できる
- [ ] NumPyとMatplotlibをimportできる
- [ ] C++出力CSVを読み込める

以上が成功すれば、線形MPCおよび自作SQP型NMPCの開発環境は完成です。

---

24. 公式参考資料

- CMake公式ダウンロード
- CMake公式チュートリアル
- VS Code公式サイト
- VS Code C++設定
- Eigen公式サイト
- Eigen 3.4.1リリース
- Eigen CMakeガイド
- OSQP公式ドキュメント
- OSQP C/C++利用方法
- OSQPソースビルド

- Python Windows向け資料
- Python venv
- Git for Windows

まとめ

本マニュアルで構築した環境は、次の開発に対応します。

```
キネマティック自転車モデル
        ↓
直線・円弧経路追従
        ↓
線形MPC
        ↓
線形時変MPC
        ↓
反復線形化MPC
        ↓
自作SQP型NMPC
        ↓
RTI-SQP
```

現在の必須構成は、次です。

```
MinGW-w64 GCC
CMake
Eigen
OSQP
C++標準ライブラリ
Python（可視化）
```

CasADi、IPOPT、acadosなどは、他方式との比較や高度な実時間NMPCへ進む段階で必要に応じて追加します。